



# Cyberscope

A *TAC Security* Company

## Audit Report

# Gravity Protocol Token

March 2026

SHA256

9c469978fc0217f0c0d0fe7a050f7ecc5acedeb649fdd09537b7b0ff81243bb7

Audited by © cyberscope

# Analysis

● Critical   ● Medium   ● Minor / Informative   ● Pass

| Severity | Code | Description             | Status     |
|----------|------|-------------------------|------------|
| ●        | ST   | Stops Transactions      | Unresolved |
| ●        | OTUT | Transfers User's Tokens | Passed     |
| ●        | ELFM | Exceeds Fees Limit      | Unresolved |
| ●        | MT   | Mints Tokens            | Passed     |
| ●        | BT   | Burns Tokens            | Passed     |
| ●        | BC   | Blacklists Addresses    | Unresolved |

# Diagnostics

● Critical ● Medium ● Minor / Informative

| Severity | Code | Description                              | Status     |
|----------|------|------------------------------------------|------------|
| ●        | BRL  | Buy Rate Limitation                      | Unresolved |
| ●        | PGA  | Potential Griefing Attack                | Unresolved |
| ●        | IBC  | Inconsistent Block Calculation           | Unresolved |
| ●        | CCR  | Contract Centralization Risk             | Unresolved |
| ●        | IDI  | Immutable Declaration Improvement        | Unresolved |
| ●        | IMWL | Incorrect Max Wallet Limitation          | Unresolved |
| ●        | IC   | Inefficient Check                        | Unresolved |
| ●        | MC   | Missing Check                            | Unresolved |
| ●        | MEE  | Missing Events Emission                  | Unresolved |
| ●        | MFI  | Missing Fee Inclusion                    | Unresolved |
| ●        | NWES | Nonconformity with ERC-20 Standard       | Unresolved |
| ●        | PLPI | Potential Liquidity Provision Inadequacy | Unresolved |
| ●        | PTRP | Potential Transfer Revert Propagation    | Unresolved |
| ●        | RRA  | Redundant Repeated Approvals             | Unresolved |

|   |      |                                            |            |
|---|------|--------------------------------------------|------------|
| ● | RSML | Redundant SafeMath Library                 | Unresolved |
| ● | RSRS | Redundant SafeMath Require Statement       | Unresolved |
| ● | SMS  | Swap Mechanism Suppression                 | Unresolved |
| ● | L02  | State Variables could be Declared Constant | Unresolved |
| ● | L04  | Conformance to Solidity Naming Conventions | Unresolved |
| ● | L15  | Local Scope Variable Shadowing             | Unresolved |
| ● | L19  | Stable Compiler Version                    | Unresolved |
| ● | L20  | Succeeded Transfer Check                   | Unresolved |

# Table of Contents

|                                         |          |
|-----------------------------------------|----------|
| <b>Analysis</b>                         | <b>1</b> |
| <b>Diagnostics</b>                      | <b>2</b> |
| <b>Table of Contents</b>                | <b>4</b> |
| <b>Risk Classification</b>              | <b>7</b> |
| <b>Review</b>                           | <b>8</b> |
| Audit Updates                           | 8        |
| Source Files                            | 8        |
| <b>Findings Breakdown</b>               | <b>9</b> |
| ST - Stops Transactions                 | 10       |
| Description                             | 10       |
| Recommendation                          | 11       |
| BC - Blacklists Addresses               | 12       |
| Description                             | 12       |
| Recommendation                          | 12       |
| ELFM - Exceeds Fees Limit               | 14       |
| Description                             | 14       |
| Recommendation                          | 14       |
| BRL - Buy Rate Limitation               | 16       |
| Description                             | 16       |
| Recommendation                          | 16       |
| PGA - Potential Griefing Attack         | 17       |
| Description                             | 17       |
| Recommendation                          | 18       |
| IBC - Inconsistent Block Calculation    | 19       |
| Description                             | 19       |
| Recommendation                          | 20       |
| CCR - Contract Centralization Risk      | 21       |
| Description                             | 21       |
| Recommendation                          | 22       |
| IDI - Immutable Declaration Improvement | 23       |
| Description                             | 23       |
| Recommendation                          | 23       |
| IMWL - Incorrect Max Wallet Limitation  | 24       |
| Description                             | 24       |
| Recommendation                          | 24       |
| IC - Inefficient Check                  | 25       |
| Description                             | 25       |
| Recommendation                          | 25       |
| MC - Missing Check                      | 26       |

|                                                  |    |
|--------------------------------------------------|----|
| Description                                      | 26 |
| Recommendation                                   | 26 |
| MEE - Missing Events Emission                    | 27 |
| Description                                      | 27 |
| Recommendation                                   | 27 |
| MFI - Missing Fee Inclusion                      | 29 |
| Description                                      | 29 |
| Recommendation                                   | 29 |
| NWES - Nonconformity with ERC-20 Standard        | 30 |
| Description                                      | 30 |
| Recommendation                                   | 30 |
| PLPI - Potential Liquidity Provision Inadequacy  | 31 |
| Description                                      | 31 |
| Recommendation                                   | 32 |
| PTRP - Potential Transfer Revert Propagation     | 33 |
| Description                                      | 33 |
| Recommendation                                   | 33 |
| RRA - Redundant Repeated Approvals               | 34 |
| Description                                      | 34 |
| Recommendation                                   | 35 |
| RSML - Redundant SafeMath Library                | 36 |
| Description                                      | 36 |
| Recommendation                                   | 37 |
| RSRS - Redundant SafeMath Require Statement      | 38 |
| Description                                      | 38 |
| Recommendation                                   | 38 |
| SMS - Swap Mechanism Suppression                 | 39 |
| Description                                      | 39 |
| Recommendation                                   | 40 |
| L02 - State Variables could be Declared Constant | 41 |
| Description                                      | 41 |
| Recommendation                                   | 41 |
| L04 - Conformance to Solidity Naming Conventions | 42 |
| Description                                      | 42 |
| Recommendation                                   | 43 |
| L15 - Local Scope Variable Shadowing             | 44 |
| Description                                      | 44 |
| Recommendation                                   | 44 |
| L19 - Stable Compiler Version                    | 45 |
| Description                                      | 45 |
| Recommendation                                   | 45 |
| L20 - Succeeded Transfer Check                   | 46 |

|                           |           |
|---------------------------|-----------|
| Description               | 46        |
| Recommendation            | 46        |
| <b>Functions Analysis</b> | <b>47</b> |
| <b>Inheritance Graph</b>  | <b>50</b> |
| <b>Flow Graph</b>         | <b>51</b> |
| <b>Summary</b>            | <b>52</b> |
| <b>Disclaimer</b>         | <b>53</b> |
| <b>About Cyberscope</b>   | <b>54</b> |

## Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

| Severity              | Likelihood / Impact of Exploitation                      |
|-----------------------|----------------------------------------------------------|
| ● Critical            | Highly Likely / High Impact                              |
| ● Medium              | Less Likely / High Impact or Highly Likely/ Lower Impact |
| ● Minor / Informative | Unlikely / Low to no Impact                              |



# Review

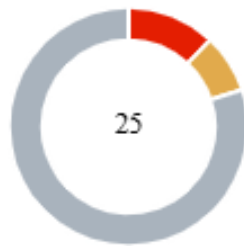
## Audit Updates

|               |             |
|---------------|-------------|
| Initial Audit | 11 Mar 2026 |
|---------------|-------------|

## Source Files

| Filename                           | SHA256                                                               |
|------------------------------------|----------------------------------------------------------------------|
| GravityProtocol_v3_audit_ready.sol | 9c469978fc0217f0c0d0fe7a050f7ecc5ace<br>deb649fdd09537b7b0ff81243bb7 |

## Findings Breakdown



|                       |    |
|-----------------------|----|
| ● Critical            | 3  |
| ● Medium              | 2  |
| ● Minor / Informative | 20 |

| Severity              | Unresolved | Acknowledged | Resolved | Other |
|-----------------------|------------|--------------|----------|-------|
| ● Critical            | 3          | 0            | 0        | 0     |
| ● Medium              | 2          | 0            | 0        | 0     |
| ● Minor / Informative | 20         | 0            | 0        | 0     |

## ST - Stops Transactions

|                    |                                                 |
|--------------------|-------------------------------------------------|
| <b>Criticality</b> | Critical                                        |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L398,437,472 |
| <b>Status</b>      | Unresolved                                      |

### Description

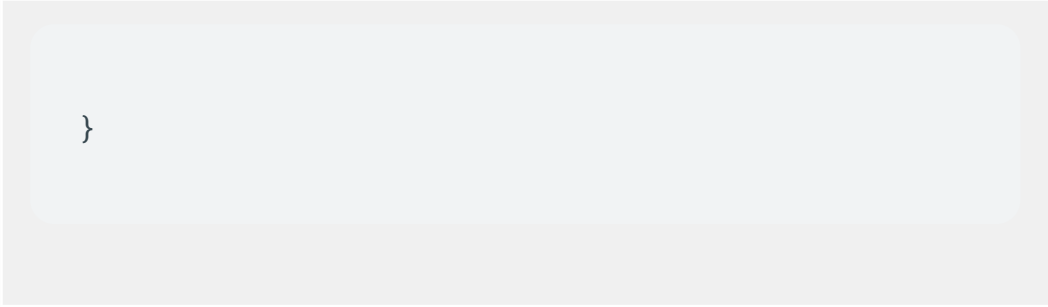
Trading is initially disabled for all users excluding the owner. The owner can enable trading for all users and once enabled, cannot disable it again.

Additionally, as described in the [BRL](#) finding the cooldown mechanism is applied to the pair address which is not excluded from the cooldown check, effectively rate-limiting all buy transactions to one purchase per cooldown period for the entire token, while the owner remains exempt from this restriction. Furthermore, transactions can be stopped as described in the [PRTP](#) and [MC](#) findings.

```
Shell
if (from != owner() && to != owner()) {
    require(tradingEnabled, "Trading not yet
enabled");
}

if (cooldownEnabled && !_isExcludedFromFee[from]
&& !_isExcludedFromFee[to]) {
    require(block.timestamp >= _lastTxTime[from] +
cooldownTime, "Cooldown active");
    _lastTxTime[from] = block.timestamp;
}

function setCooldownTime(uint256 _seconds)
external onlyOwner {
    require(_seconds <= 300, "Cooldown max 5
minutes");
    cooldownTime = _seconds;
}
```



## Recommendation

The team should follow the recommendations of the `PTRP` and `BRL` and `MC` findings. The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

### Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

### Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

## BC - Blacklists Addresses

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Critical                                |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L434 |
| <b>Status</b>      | Unresolved                              |

### Description

For the first 48 hours after trading is enabled, the contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `setBlacklist` function.

```
Shell
function _transfer(address from, address to,
uint256 amount) private {
    ...
    require(!isBlacklisted[from] &&
!isBlacklisted[to], "Blacklisted address");
    ...
}
```

### Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

#### Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.

- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

## ELFM - Exceeds Fees Limit

|             |                                                 |
|-------------|-------------------------------------------------|
| Criticality | Medium                                          |
| Location    | GravityProtocol_v3_audit_ready.sol#L171,178,582 |
| Status      | Unresolved                                      |

### Description

During the anti-snipe period, all buyers face a 99% tax applied in `_tokenTransferSnipe()`. The owner may extend this period by calling the `setAntiBotBlocks()` function with a higher value, up to 20 blocks.

```
Shell
uint256 public antiBotBlocks = 10;

uint256 public snipeTaxRate = 99;

function _tokenTransferSnipe(address sender,
address recipient, uint256 amount) private {
    ...
    uint256 taxAmount =
amount.mul(snipeTaxRate).div(100);
    ...
}
```

### Recommendation

After the first 20 block pass the fees can be up to 25% which is the maximum accepted fee value. The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

### Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

### Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.



## BRL - Buy Rate Limitation

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Critical                                |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L472 |
| <b>Status</b>      | Unresolved                              |

### Description

The PancakeSwap pair address is not excluded from the cooldown check. Since all buy transactions originate from the pair address, the cooldown timer is shared across all buyers, effectively limiting the entire token to one buy transaction per cooldown period. This restriction does not apply to the owner, who is excluded from the cooldown check.

Shell

```
if (cooldownEnabled && !_isExcludedFromFee[from]
    && !_isExcludedFromFee[to]) {
    require(block.timestamp >= _lastTxTime[from] +
cooldownTime, "Cooldown active");
    _lastTxTime[from] = block.timestamp;
}
```

### Recommendation

The team is advised to exclude the pair address from the cooldown mechanism to allow individual buy transactions to operate independently without sharing a global cooldown timer.

## PGA - Potential Griefing Attack

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Medium                                  |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L472 |
| <b>Status</b>      | Unresolved                              |

### Description

The contract includes functionality designed to enforce specific conditions on transactions. However, this design is vulnerable to griefing attacks, where malicious actors can exploit the contract's logic to interfere with legitimate user operations.

In this case, the contract enforces transactional limits based on on-chain activity. A third party could strategically interact with the contract's state to disrupt normal user operations, resulting in failed transactions or unintended behavior. Specifically, a third party user could transfer tokens to the pair contract to restrict other users of buying tokens.

Such griefing attacks could undermine the contract's usability and obstruct legitimate user operations.

Shell

```
if (cooldownEnabled && !_isExcludedFromFee[from]
    && !_isExcludedFromFee[to]) {
    require(block.timestamp >= _lastTxTime[from] +
cooldownTime, "Cooldown active");
    _lastTxTime[from] = block.timestamp;
}
```

## Recommendation

The team is advised to review the transfer mechanism to ensure that all legitimate operations are processed as intended. This will help maintain the integrity of user activities and strengthen trust in the system.

## IBC - Inconsistent Block Calculation

|                    |                                             |
|--------------------|---------------------------------------------|
| <b>Criticality</b> | Minor / Informative                         |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L171,442 |
| <b>Status</b>      | Unresolved                                  |

### Description

The contract implements an anti-snipe mechanism that applies a 99% tax during the first blocks after trading is enabled. The variable `antiBotBlocks` is set to 10, suggesting that the protection should cover 10 blocks. However, the conditional statement uses a less-than-or-equal operator (`<=`) instead of strict less-than (`<`). As a result, the anti-snipe tax is applied for 11 blocks instead of the expected 10, creating an inconsistency between the variable value and the actual behavior.

```
Shell
uint256 public antiBotBlocks = 10;

bool isSnipePeriod = (
    tradingEnabled &&
    block.number <= tradingEnabledBlock +
    antiBotBlocks &&
    from == pancakePair &&
    to != address(pancakeRouter) &&
    to != owner() &&
    !_isExcludedFromFee[to]
);
```

## Recommendation

The team is advised to ensure that the anti-snipe duration is consistent with the variable value and intended documentation. The conditional operator should accurately reflect the desired number of protected blocks to prevent any discrepancy between expected and actual behavior.

## CCR - Contract Centralization Risk

|                    |                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------|
| <b>Criticality</b> | Minor / Informative                                                                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L320,331,344,348,354,362,368,377,383,389,403,408,413 |
| <b>Status</b>      | Unresolved                                                                              |

### Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Shell

```
function excludeFromReward(address account) public
onlyOwner {...}
function includeInReward(address account) external
onlyOwner {...}
function excludeFromFee(address account) public
onlyOwner {...}
function includeInFee(address account) public
onlyOwner {...}
function enableTrading() external onlyOwner {...}
function setBlacklist(address account, bool value)
external onlyOwner {...}
function setProtocolFundWallet(address payable
newWallet) external onlyOwner {...}
function setMaxTxAmount(uint256 amount) external
onlyOwner {...}
function setMaxWalletAmount(uint256 amount)
external onlyOwner {...}
function setSwapAndLiquifyEnabled(bool _enabled)
public onlyOwner {...}
function setCooldownEnabled(bool _enabled)
```

```
external onlyOwner {...}
function setCooldownTime(uint256 _seconds)
external onlyOwner {...}
function setAntiBotBlocks(uint256 _blocks)
external onlyOwner {...}
function setEarlySellerExtraFee(uint256 _fee)
external onlyOwner {...}

function setNumTokensSellToAddToLiquidity(uint256
amount) external onlyOwner {...}
```

## Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

## IDI - Immutable Declaration Improvement

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Minor / Informative                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L232 |
| <b>Status</b>      | Unresolved                              |

### Description

The contract declares state variables that their value is initialized once in the constructor and are not modified afterwards. The `immutable` is a special declaration for this kind of state variable that saves gas when it is defined.

```
Shell
pancakePair
```

### Recommendation

By declaring a variable as immutable, the Solidity compiler is able to make certain optimizations. This can reduce the amount of storage and computation required by the contract, and make it more gas-efficient.



## IMWL - Incorrect Max Wallet Limitation

|             |                                         |
|-------------|-----------------------------------------|
| Criticality | Minor / Informative                     |
| Location    | GravityProtocol_v3_audit_ready.sol#L468 |
| Status      | Unresolved                              |

### Description

The `_transfer` function has a requirement that checks if the balance of the receiver plus the amount sent is less than or equal to the max wallet amount. However the actual amount received will be less due to the fees applied. As a result the restriction is more strict than intended.

```
Shell
require(balanceOf(to).add(amount) <=
maxWalletAmount, "Exceeds max wallet amount");
```

### Recommendation

The team is advised to enforce max wallet amount restriction based on the amount transferred to the receiver after the fee calculation.

## IC - Inefficient Check

|                    |                                             |
|--------------------|---------------------------------------------|
| <b>Criticality</b> | Minor / Informative                         |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L225,370 |
| <b>Status</b>      | Unresolved                                  |

### Description

The contract tries to ensure that `protocolFundWallet` is not the deployer or the current owner of the contract; however, an external user may have a different account, essentially making the check inefficient.

```
Shell
require(_protocolFundWallet != _msgSender(),
"Protocol fund wallet must differ from deployer");
...
require(newWallet != owner(), "Protocol fund
wallet must differ from owner");
```

### Recommendation

The team is advised to remove these restrictions as they can easily be bypassed in order to increase gas optimization and efficiency.

## MC - Missing Check

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Minor / Informative                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L537 |
| <b>Status</b>      | Unresolved                              |

### Description

The contract is processing variables that have not been properly sanitized and checked that they form the proper shape. These variables may produce vulnerability issues.

Specifically a check is missing to ensure that the contract safely returns if `tokensToSwap` are zero during the `_swapAndLiquify` function. If the contract does not return early a division by 0 will revert the entire transaction.

```
Shell
function _swapAndLiquify(uint256
contractTokenBalance) private lockTheSwap {
    ...
    uint256 liquidityETH =
newBalance.mul(half).div(tokensToSwap);
    ...
}
```

### Recommendation

The team is advised to properly check the variables according to the required specifications.

## MEE - Missing Events Emission

|                    |                                                                 |
|--------------------|-----------------------------------------------------------------|
| <b>Criticality</b> | Minor / Informative                                             |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L320,331,344,348,394,398,403 |
| <b>Status</b>      | Unresolved                                                      |

### Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

Shell

```
function excludeFromReward(address account) public  
  onlyOwner  
function includeInReward(address account) external  
  onlyOwner  
function excludeFromFee(address account) public  
  onlyOwner  
function includeInFee(address account) public  
  onlyOwner  
function setCooldownEnabled(bool _enabled)  
  external onlyOwner  
  
function setCooldownTime(uint256 _seconds)  
  external onlyOwner
```

### Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such

as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

## MFI - Missing Fee Inclusion

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Minor / Informative                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L368 |
| <b>Status</b>      | Unresolved                              |

### Description

The owner is able to change the `protocolFundWallet` by calling the `setProtocolFundWallet` function. The function excludes the new address from fees but it does not include the old address.

```
Shell
function setProtocolFundWallet(address payable
newWallet) external onlyOwner {
    require(newWallet != address(0), "Cannot be
zero address");
    require(newWallet != owner(), "Protocol fund
wallet must differ from owner");
    address oldWallet = protocolFundWallet;
    protocolFundWallet = newWallet;
    _isExcludedFromFee[newWallet] = true;
    emit ProtocolFundWalletUpdated(oldWallet,
newWallet);
}
```

### Recommendation

The team is advised to include the old address into fees after the `protocolFundWallet` update.

## NWES - Nonconformity with ERC-20 Standard

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Minor / Informative                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L430 |
| <b>Status</b>      | Unresolved                              |

### Description

The contract does not fully conform to the ERC20 Standard. Specifically, according to the standard, transfers of 0 values must be treated as normal transfers and fire the Transfer event. However, the contract implements a conditional check that prohibits transfers of 0 values. This discrepancy between the contract's implementation and the ERC20 standard may lead to inconsistencies and incompatibilities with other contracts.

Shell

```
function _transfer(address from, address to,  
uint256 amount) private {  
    ...  
    require(amount > 0, "Transfer amount must be  
greater than zero");  
    ...  
}
```

### Recommendation

The incorrect implementation of the ERC20 standard could potentially lead to problems when interacting with the contract, as other contracts or applications that expect the ERC20 interface may not behave as expected. The team is advised to review and revise the implementation of the transfer mechanism to ensure full compliance with the ERC20 standard. <https://eips.ethereum.org/EIPS/eip-20>.

## PLPI - Potential Liquidity Provision Inadequacy

|                    |                                             |
|--------------------|---------------------------------------------|
| <b>Criticality</b> | Minor / Informative                         |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L558,570 |
| <b>Status</b>      | Unresolved                                  |

### Description

The contract operates under the assumption that liquidity is consistently provided to the pair between the contract's token and the native currency. However, there is a possibility that liquidity is provided to a different pair. This inadequacy in liquidity provision in the main pair could expose the contract to risks. Specifically, during eligible transactions, where the contract attempts to swap tokens with the main pair, a failure may occur if liquidity has been added to a pair other than the primary one. Consequently, transactions triggering the swap functionality will result in a revert.

Shell

```
pancakeRouter.swapExactTokensForETHSupportingFeeOnTransferTokens(tokenAmount, 0, path, address(this),
block.timestamp);
function _swapTokensForEth(uint256 tokenAmount) private
{
    ...

    pancakeRouter.addLiquidityETH{value:
ethAmount}(address(this), tokenAmount, 0, 0, DEAD,
block.timestamp);
```



## Recommendation

The team is advised to implement a runtime mechanism to check if the pair has adequate liquidity provisions. This feature allows the contract to omit token swaps if the pair does not have adequate liquidity provisions, significantly minimizing the risk of potential failures.

Furthermore, the team could ensure the contract has the capability to switch its active pair in case liquidity is added to another pair.

Additionally, the contract could be designed to tolerate potential reverts from the swap functionality, especially when it is a part of the main transfer flow. This can be achieved by executing the contract's token swaps in a non-reversible manner, thereby ensuring a more resilient and predictable operation.

## PTRP - Potential Transfer Revert Propagation

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Minor / Informative                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L545 |
| <b>Status</b>      | Unresolved                              |

### Description

The contract sends funds to a `protocolFundWallet` as part of the transfer flow. This address can either be a wallet address or a contract. If the address belongs to a contract then it may revert from incoming payment. As a result, the error will propagate to the token's contract and revert the transfer.

```
Shell
if (protocolFundETH > 0) {
    (bool success, ) =
    protocolFundWallet.call{value:
    protocolFundETH}("");
    require(success, "BNB transfer to protocol
    fund failed");
}
```

### Recommendation

The contract should tolerate the potential revert from the underlying contracts when the interaction is part of the main transfer flow. This could be achieved by not allowing set contract addresses or by sending the funds in a non-revertable way.

## RRA - Redundant Repeated Approvals

|                    |                                             |
|--------------------|---------------------------------------------|
| <b>Criticality</b> | Minor / Informative                         |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L556,568 |
| <b>Status</b>      | Unresolved                                  |

### Description

The contract is designed to `approve` token transfers during the contract's operation by calling the `_approve` function before specific operations. This approach results in additional gas costs since the approval process is repeated for every operation execution, leading to inefficiencies and increased transaction expenses.

Shell

```
function _swapTokensForEth(uint256 tokenAmount) private
{
    ...
    _approve(address(this), address(pancakeRouter),
tokenAmount);

    pancakeRouter.swapExactTokensForETHSupportingFeeOnTransf
erTokens(tokenAmount, 0, path, address(this),
block.timestamp);
}

function _addLiquidity(uint256 tokenAmount, uint256
ethAmount) private {
    _approve(address(this), address(pancakeRouter),
tokenAmount);
    pancakeRouter.addLiquidityETH{value:
ethAmount}(address(this), tokenAmount, 0, 0, DEAD,
block.timestamp);
}
```

## Recommendation

Since the approved address is a trusted third-party source, it is recommended to optimize the contract by approving the maximum amount of tokens once in the initial set of the variable, rather than before each operation. This change will reduce the overall gas consumption and improve the efficiency of the contract.

## RSML - Redundant SafeMath Library

|                    |                                        |
|--------------------|----------------------------------------|
| <b>Criticality</b> | Minor / Informative                    |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L60 |
| <b>Status</b>      | Unresolved                             |

### Description

SafeMath is a popular Solidity library that provides a set of functions for performing common arithmetic operations in a way that is resistant to integer overflows and underflows.

Starting with Solidity versions that are greater than or equal to 0.8.0, the arithmetic operations revert to underflow and overflow. As a result, the native functionality of the Solidity operations replaces the SafeMath library. Hence, the usage of the SafeMath library adds complexity, overhead and increases gas consumption unnecessarily in cases where the explanatory error message is not used.

Shell

```
library SafeMath {...}
```

## Recommendation

The team is advised to remove the SafeMath library in cases where the revert error message is not used. Since the version of the contract is greater than `0.8.0` then the pure Solidity arithmetic operations produce the same result.

If the previous functionality is required, then the contract could exploit the `unchecked { ... }` statement.

Read more about the breaking change on

<https://docs.soliditylang.org/en/stable/080-breaking-changes.html#solidity-v0-8-0-breaking-changes>.

## RSRS - Redundant SafeMath Require Statement

|                    |                                        |
|--------------------|----------------------------------------|
| <b>Criticality</b> | Minor / Informative                    |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L63 |
| <b>Status</b>      | Unresolved                             |

### Description

The contract utilizes a `require` statement within the `add` function aiming to prevent overflow errors. This function is designed based on the SafeMath library's principles. In Solidity version 0.8.0 and later, arithmetic operations revert to overflow and underflow, making the overflow check within the function redundant. This redundancy could lead to extra gas costs and increased complexity without providing additional security.

Shell

```
function add(uint256 a, uint256 b) internal pure
returns (uint256) {
    uint256 c = a + b;
    require(c >= a, "SafeMath: addition
overflow");
    return c;
}
```

### Recommendation

It is recommended to remove the `require` statement from the `add` function since the contract is using a Solidity pragma version equal to or greater than 0.8.0. By doing so, the contract will leverage the built-in overflow and underflow checks provided by the Solidity language itself, simplifying the code and reducing gas consumption. This change will uphold the contract's integrity in handling arithmetic operations while optimizing for efficiency and cost-effectiveness.

## SMS - Swap Mechanism Suppression

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Minor / Informative                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L490 |
| <b>Status</b>      | Unresolved                              |

### Description

The contract implements a swap-and-liquify mechanism that depends on the relationship between `maxTxAmount` and `numTokensSellToAddToLiquidity`. However, due to the logic that caps the contract's token balance to `maxTxAmount` before checking whether the swap threshold is reached, the effective condition for triggering swaps becomes dependent on the owner-controlled parameters. If the owner sets `maxTxAmount` lower than `numTokensSellToAddToLiquidity`, the threshold condition can never be satisfied, even if the contract accumulates a large number of fee tokens. As a result, the swap-and-liquify process can be permanently disabled, causing tokens collected as fees to accumulate in the contract without being converted to liquidity or protocol funds, which alters the expected tokenomics and can lead to unpredictable future market behavior when swaps are re-enabled.

```
Shell
uint256 contractTokenBalance =
    balanceOf(address(this));
if (contractTokenBalance >= maxTxAmount) {
    contractTokenBalance = maxTxAmount;
}

bool overMinTokenBalance = contractTokenBalance >=
    numTokensSellToAddToLiquidity;
if (
    overMinTokenBalance &&
    !inSwapAndLiquify &&
    from != pancakePair &&
    swapAndLiquifyEnabled
```



```
) {  
    contractTokenBalance =  
    numTokensSellToAddToLiquidity;  
    _swapAndLiquify(contractTokenBalance);  
}
```

## Recommendation

The team is advised to ensure that the swap trigger condition is independent of the capped transaction amount or to enforce a constraint that guarantees `maxTxAmount` is always greater than or equal to `numTokensSellToAddToLiquidity`. This can be achieved by validating parameter updates or by restructuring the logic so that the threshold check uses the actual contract balance before applying any caps. This will help preserve the intended functionality of the swap-and-liquify mechanism and prevent the owner from unintentionally or deliberately disabling it.

## L02 - State Variables could be Declared Constant

|                    |                                             |
|--------------------|---------------------------------------------|
| <b>Criticality</b> | Minor / Informative                         |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L139,172 |
| <b>Status</b>      | Unresolved                                  |

### Description

State variables can be declared as constant using the constant keyword. This means that the value of the state variable cannot be changed after it has been set. Additionally, the constant variables decrease gas consumption of the corresponding transaction.

```
Shell
uint256 private _tTotal = 1_000_000_000_000_000 *
10**9

uint256 public snipeTaxRate = 99
```

### Recommendation

Constant state variables can be useful when the contract wants to ensure that the value of a state variable cannot be changed by any function in the contract. This can be useful for storing values that are important to the contract's behavior, such as the contract's address or the maximum number of times a certain function can be called. The team is advised to add the constant keyword to state variables that never change.

## L04 - Conformance to Solidity Naming Conventions

|                    |                                                                            |
|--------------------|----------------------------------------------------------------------------|
| <b>Criticality</b> | Minor / Informative                                                        |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L40,133,134,135,262,389,394,398,403,408 |
| <b>Status</b>      | Unresolved                                                                 |

### Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX\_VALUE, ERROR\_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

Shell

```
function WETH() external pure returns (address);
string private constant _name = "Gravity Protocol"
string private constant _symbol = "GRAV"
uint8 private constant _decimals = 9
address _owner
bool _enabled
uint256 _seconds
uint256 _blocks
```

```
uint256 _fee
```

## Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

## L15 - Local Scope Variable Shadowing

|                    |                                             |
|--------------------|---------------------------------------------|
| <b>Criticality</b> | Minor / Informative                         |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L262,423 |
| <b>Status</b>      | Unresolved                                  |

### Description

Local scope variable shadowing occurs when a local variable with the same name as a variable in an outer scope is declared within a function or code block. When this happens, the local variable "shadows" the outer variable, meaning that it takes precedence over the outer variable within the scope in which it is declared.

```
Shell
address _owner
```

### Recommendation

It's important to be aware of shadowing when working with local variables, as it can lead to confusion and unintended consequences if not used correctly. It's generally a good idea to choose unique names for local variables to avoid shadowing outer variables and causing confusion.

## L19 - Stable Compiler Version

|                    |                                       |
|--------------------|---------------------------------------|
| <b>Criticality</b> | Minor / Informative                   |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L2 |
| <b>Status</b>      | Unresolved                            |

### Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

Shell

```
pragma solidity ^0.8.19;
```

### Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

## L20 - Succeeded Transfer Check

|                    |                                         |
|--------------------|-----------------------------------------|
| <b>Criticality</b> | Minor / Informative                     |
| <b>Location</b>    | GravityProtocol_v3_audit_ready.sol#L811 |
| <b>Status</b>      | Unresolved                              |

### Description

According to the ERC20 specification, the transfer methods should be checked if the result is successful. Otherwise, the contract may wrongly assume that the transfer has been established.

Shell

```
ERC20(tokenAddress).transfer(owner(), amount);
```

### Recommendation

The contract should check if the result of the transfer methods is successful. The team is advised to check the SafeERC20 library from the [Openzeppelin library](#).

# Functions Analysis

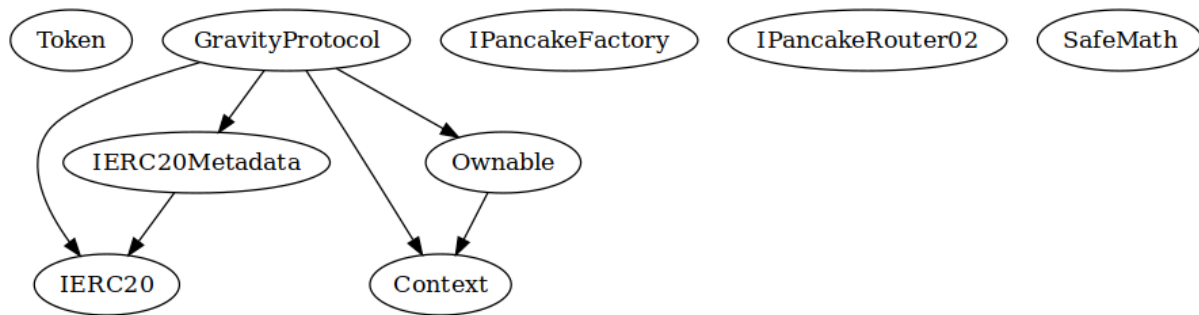
| Contract        | Type                 | Bases                                                 |            |           |
|-----------------|----------------------|-------------------------------------------------------|------------|-----------|
|                 | Function Name        | Visibility                                            | Mutability | Modifiers |
|                 |                      |                                                       |            |           |
| GravityProtocol | Implementation       | Context,<br>IERC20,<br>IERC20Meta<br>data,<br>Ownable |            |           |
|                 |                      | Public                                                | ✓          | -         |
|                 | name                 | Public                                                |            | -         |
|                 | symbol               | Public                                                |            | -         |
|                 | decimals             | Public                                                |            | -         |
|                 | totalSupply          | Public                                                |            | -         |
|                 | balanceOf            | Public                                                |            | -         |
|                 | transfer             | Public                                                | ✓          | -         |
|                 | allowance            | Public                                                |            | -         |
|                 | approve              | Public                                                | ✓          | -         |
|                 | transferFrom         | Public                                                | ✓          | -         |
|                 | increaseAllowance    | Public                                                | ✓          | -         |
|                 | decreaseAllowance    | Public                                                | ✓          | -         |
|                 | tokenFromReflection  | Public                                                |            | -         |
|                 | reflectionFromToken  | Public                                                |            | -         |
|                 | totalFees            | Public                                                |            | -         |
|                 | isExcludedFromReward | Public                                                |            | -         |
|                 | isExcludedFromFee    | Public                                                |            | -         |
|                 | excludeFromReward    | Public                                                | ✓          | onlyOwner |



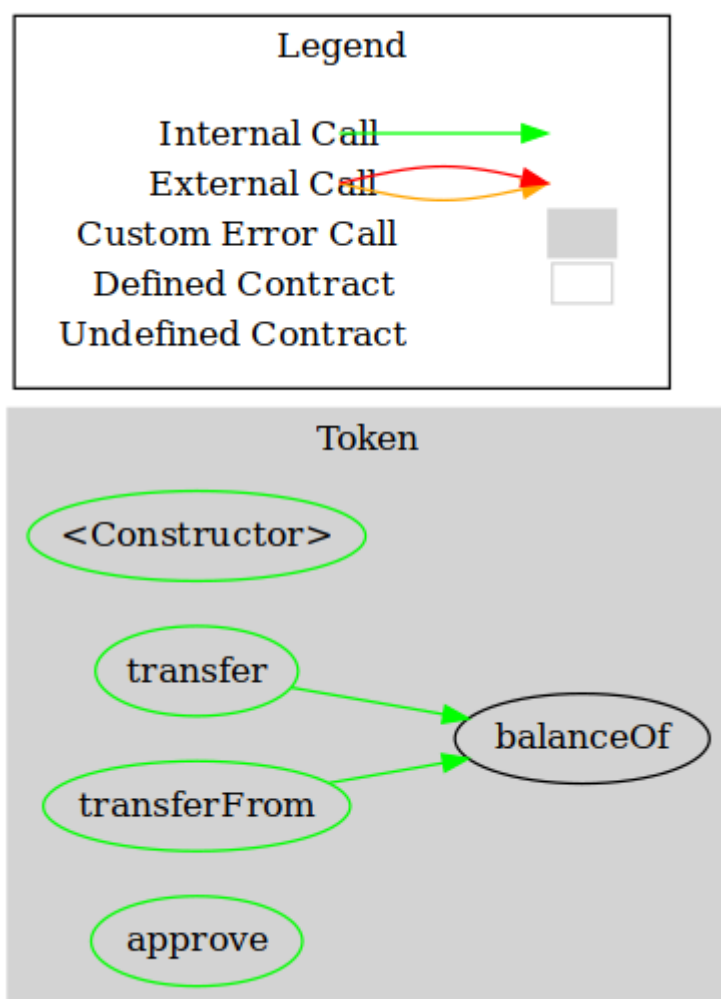
|  |                                  |          |   |             |
|--|----------------------------------|----------|---|-------------|
|  | includeInReward                  | External | ✓ | onlyOwner   |
|  | excludeFromFee                   | Public   | ✓ | onlyOwner   |
|  | includeInFee                     | Public   | ✓ | onlyOwner   |
|  | enableTrading                    | External | ✓ | onlyOwner   |
|  | setBlacklist                     | External | ✓ | onlyOwner   |
|  | setProtocolFundWallet            | External | ✓ | onlyOwner   |
|  | setMaxTxAmount                   | External | ✓ | onlyOwner   |
|  | setMaxWalletAmount               | External | ✓ | onlyOwner   |
|  | setSwapAndLiquifyEnabled         | Public   | ✓ | onlyOwner   |
|  | setCooldownEnabled               | External | ✓ | onlyOwner   |
|  | setCooldownTime                  | External | ✓ | onlyOwner   |
|  | setAntiBotBlocks                 | External | ✓ | onlyOwner   |
|  | setEarlySellerExtraFee           | External | ✓ | onlyOwner   |
|  | setNumTokensSellToAddToLiquidity | External | ✓ | onlyOwner   |
|  | _approve                         | Private  | ✓ |             |
|  | _transfer                        | Private  | ✓ |             |
|  | _swapAndLiquify                  | Private  | ✓ | lockTheSwap |
|  | _swapTokensForEth                | Private  | ✓ |             |
|  | _addLiquidity                    | Private  | ✓ |             |
|  | _tokenTransferSnipe              | Private  | ✓ |             |
|  | _tokenTransfer                   | Private  | ✓ |             |
|  | _transferStandard                | Private  | ✓ |             |
|  | _transferToExcluded              | Private  | ✓ |             |
|  | _transferFromExcluded            | Private  | ✓ |             |
|  | _transferBothExcluded            | Private  | ✓ |             |

|  |                         |          |         |           |
|--|-------------------------|----------|---------|-----------|
|  | _getValues              | Private  |         |           |
|  | _getTValues             | Private  |         |           |
|  | _getRate                | Private  |         |           |
|  | _getCurrentSupply       | Private  |         |           |
|  | _takeLiquidity          | Private  | ✓       |           |
|  | _takeBurn               | Private  | ✓       |           |
|  | _reflectFee             | Private  | ✓       |           |
|  | _removeAllFee           | Private  | ✓       |           |
|  | _restoreAllFee          | Private  | ✓       |           |
|  | emergencyWithdrawBNB    | External | ✓       | onlyOwner |
|  | emergencyWithdrawTokens | External | ✓       | onlyOwner |
|  |                         | External | Payable | -         |

## Inheritance Graph



## Flow Graph



## Summary

Gravity Protocol contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions, manipulate the fees and massively blacklist addresses. A multi-wallet signing pattern will provide security against potential hacks. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats.

## Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

# About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

[cyberscope.io](https://cyberscope.io)